STUDY SESSION OPTIMIZER

Final Project Report

CSE1021 – Problem Solving and Programming

Python Console Application

Project Type	Rule-based study planning and progress tracking system
Implementation	Python
Interface	Console / command-line
Core Logic	Priority scoring, sorting, completion tracking and search

Prepared for project submission

1. Introduction

The Study Session Optimizer is a rule-based console application for organizing study topics. The system accepts a subject, topic name, difficulty, deadline and estimated study time. It calculates a priority score, suggests an order for pending topics, tracks completion, supports topic search and displays progress information.

2. Problem Statement

Students may have several topics with different difficulty levels, deadlines and expected study times. Without a simple prioritization method, deciding what to study first can become difficult. This project provides a lightweight rule-based solution that converts these factors into a priority score and presents a suggested study order.

3. Objectives

• Record and validate study topics.

• Calculate a priority score from difficulty, deadline and study time.

• Generate a suggested study order for pending topics.

• Mark topics as completed.

• Search by topic name or subject.

• Display progress information.

4. Scope and Requirements

Category	Requirement
Input	Subject, topic name, difficulty, deadline and estimated study time
Processing	Priority calculation using rule-based scoring
Planning	Pending topics sorted by priority
Tracking	Completion status and progress percentage
Search	Search by topic name or subject
Validation	Difficulty, date and study-time validation
Output	Formatted console menus, topic details and summaries

5. Priority Algorithm

The priority score is the sum of three rule-based components: difficulty points, deadline points and study-time points.

Factor	Rule	Points
Difficulty	Easy / Medium / Hard	1 / 2 / 3
Deadline	≤2 / ≤5 / ≤10 / ≤20 / >20 days	5 / 4 / 3 / 2 / 1
Study time	≥4 / ≥2 / <2 hours	3 / 2 / 1

Factor	Rule	Points
Total	Sum of the three components	Priority Score

Example: A hard topic due within two days with an estimated study time of four hours receives 3 + 5 + 3 = 11 points. Higher scores are placed earlier in the suggested study order.

6. System Architecture and Design

The project is organized around separate responsibilities so that input handling, priority calculation, topic management, planning, progress tracking and search can be maintained independently.

User / Console	→	Main Menu / Controller
Main Menu / Controller	→	Topic Manager
Topic Manager	→	Priority Calculator
Main Menu / Controller	→	Study Planner
Main Menu / Controller	→	Progress Tracker
Main Menu / Controller	→	Search Manager
Supporting layer	→	Input Validator

6.1 Project Structure

```
Study-Session-Optimizer/
■■■ main.py
■■■ topic_manager.py
■■■ priority_calculator.py
■■■ study_planner.py
■■■ progress_tracker.py
■■■ search_manager.py
■■■ input_validator.py
■■■ tests/
■ ■■■ test_project.py
■■■ README.md
■■■ statement.md
■■■ requirements.txt
```

The application data is held in memory during execution using topic records. The current implementation does not persist topics after the program exits.

7. Implementation Details

Module	Responsibility
main.py	Console menu and orchestration of the application
topic_manager.py	Creation and display of topic records
priority_calculator.py	Priority score and priority level calculation
study_planner.py	Filtering pending topics and generating study order
progress_tracker.py	Completion status and progress summary
search_manager.py	Search by topic name or subject

Module	Responsibility
input_validator.py	Validation of difficulty, date, study time and selections
tests/test_project.py	Automated tests for core functionality

8. Application Workflow

1. Start the application.
2. Select an operation from the seven-item menu.
3. For Add Topic, enter the required topic information.
4. Validate the input values.
5. Calculate the priority score and store the topic.
6. Use Suggested Study Order to sort pending topics by priority.
7. Use Mark Topic as Completed to update completion status.
8. Use Search Topic to locate topics by name or subject.
9. Use Progress Summary to display completed, pending and percentage progress.
10. Exit the application.

9. Screenshots and Results

The following screenshots are genuine execution captures supplied for the project submission. They document the console application, its priority calculation, suggested ordering, topic details, completion tracking and progress output.

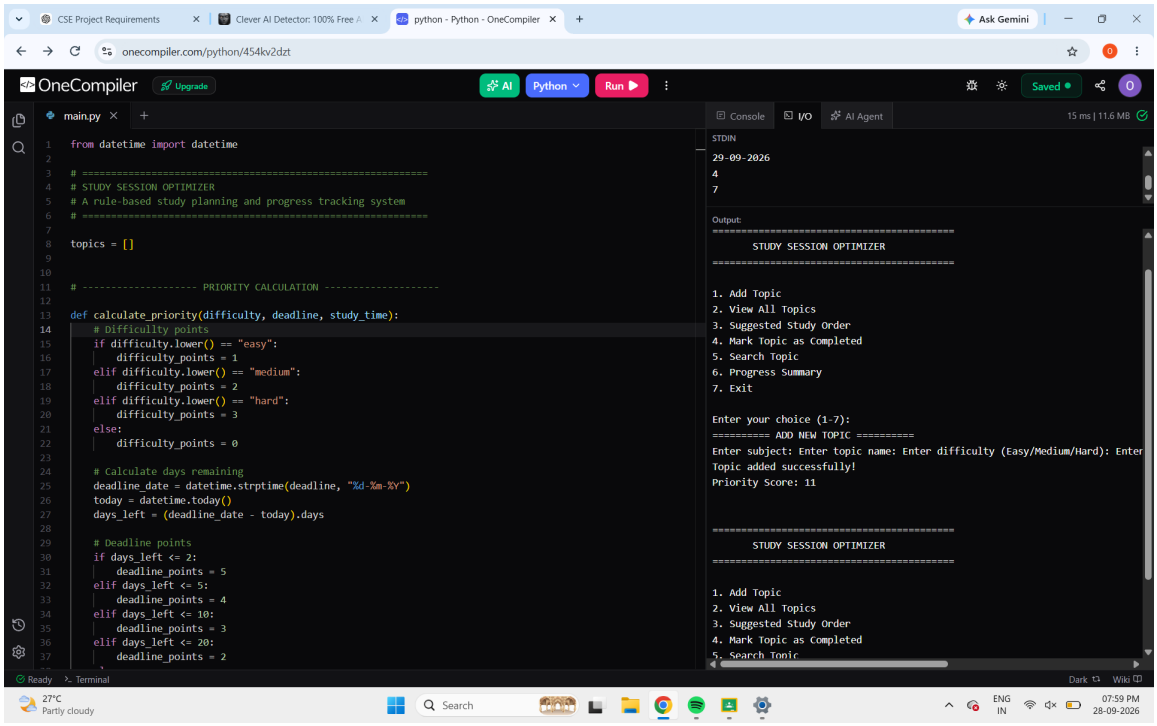


Figure 1. Program execution showing the main menu, topic entry and a calculated priority score.

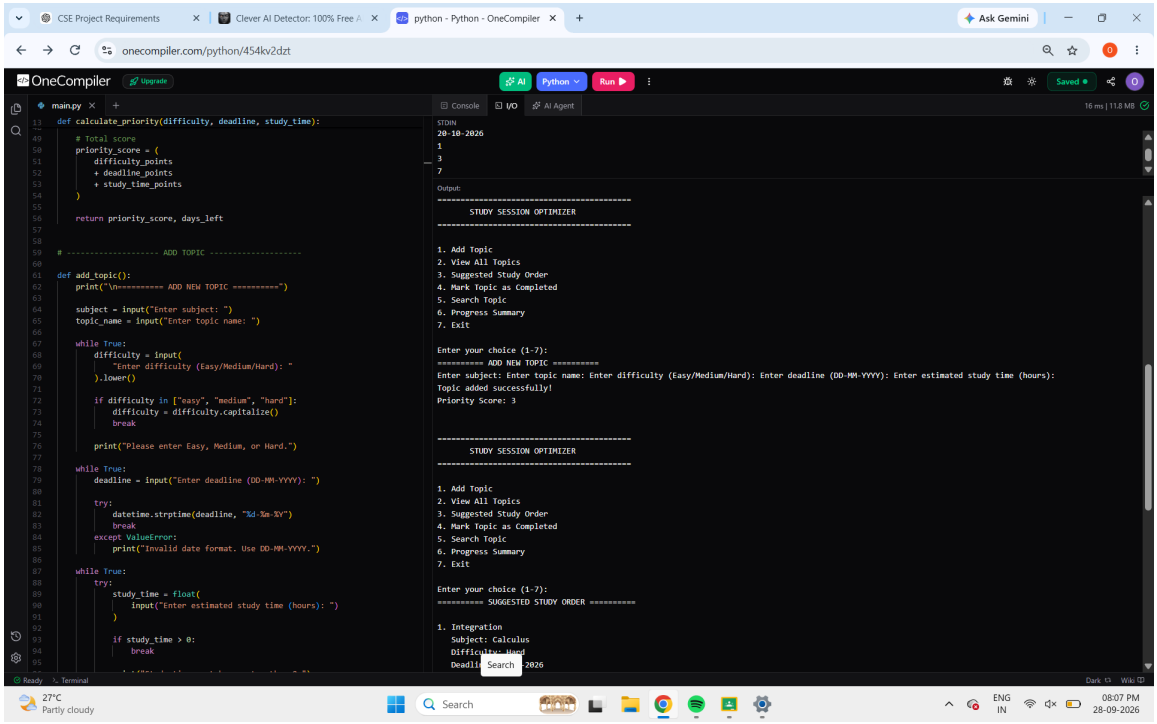


Figure 2. Program execution showing topic input and the Suggested Study Order feature.

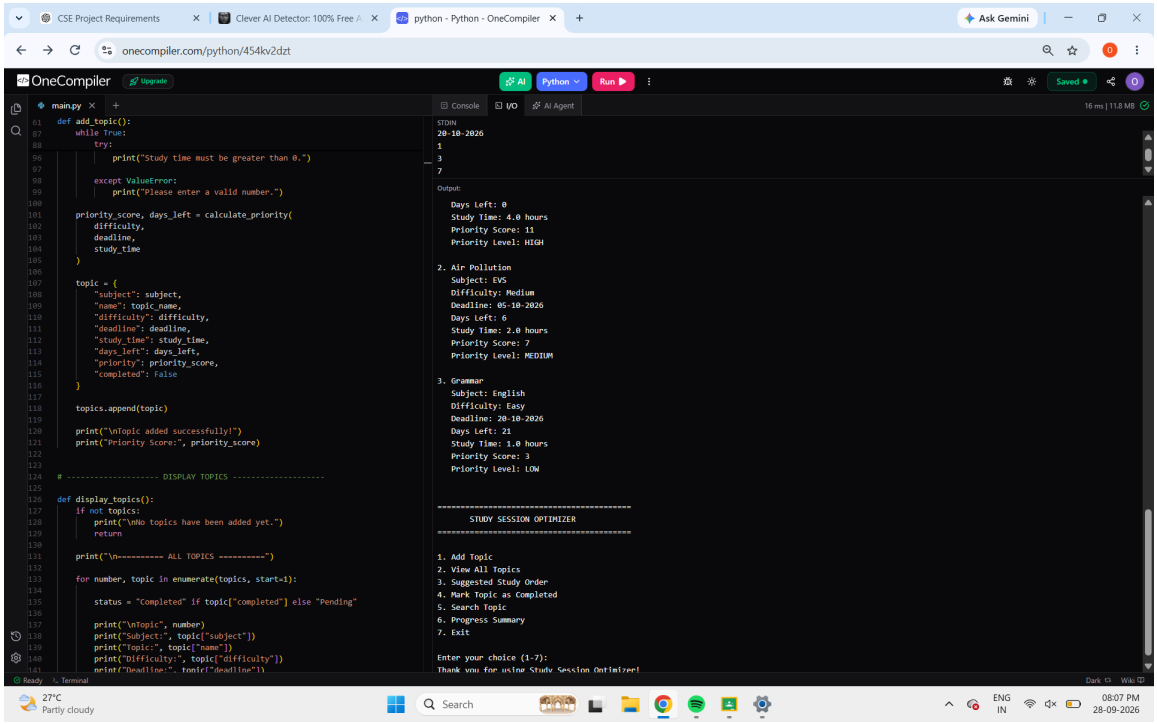


Figure 3. Program execution showing the detailed suggested order and topic information.

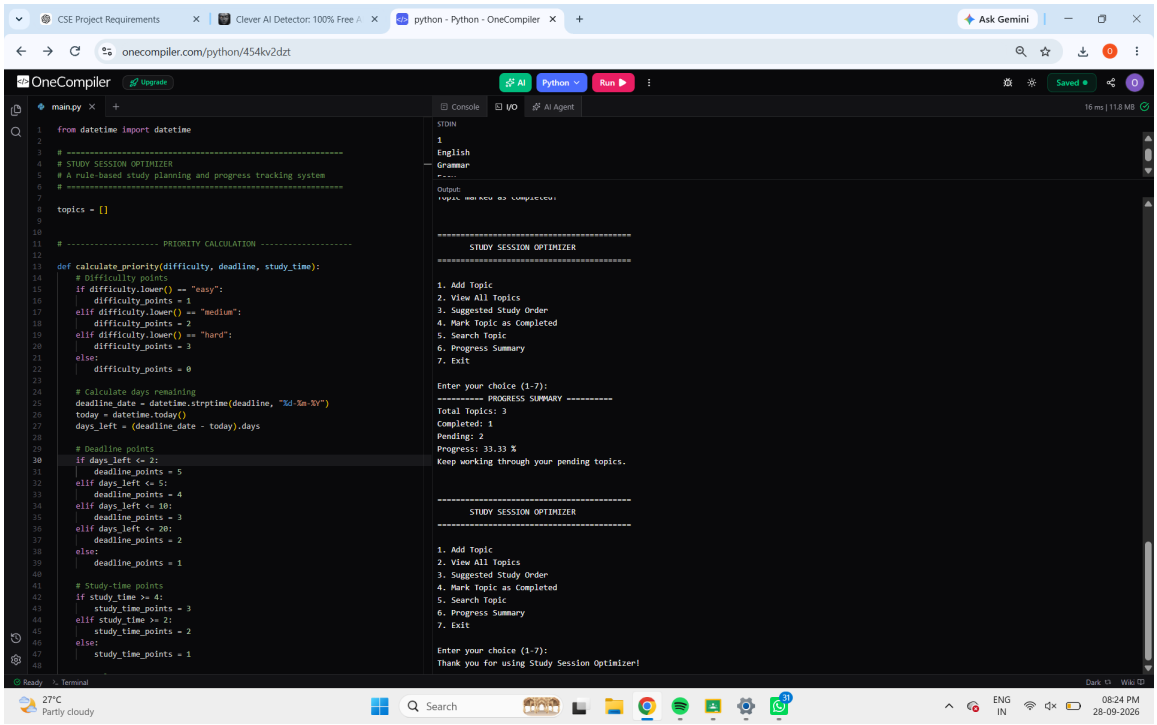


Figure 4. Program execution showing the Progress Summary with completed, pending and percentage progress.

10. Testing

ID	Test	Expected Result
TC-01	Add Topic	Topic added and priority displayed
TC-02	View Topics	Stored topics displayed
TC-03	Suggested Order	Pending topics sorted by priority
TC-04	Mark Completed	Selected topic becomes completed
TC-05	Progress	Counts and percentage displayed
TC-06	Search	Matching topic/subject displayed
TC-07	Invalid Input	Invalid value rejected and re-entered

10.1 Testing Approach

Testing covers the main user-facing operations and input validation. The supplied project also includes a tests folder containing automated checks for core functions. Console execution was additionally verified through the attached screenshots.

11. Design Rationale

• **Rule-based prioritization:** A transparent point system makes the suggested order easy to understand.

• **Modular responsibilities:** Separate modules reduce coupling and make individual functions easier to test and maintain.

• **Console interface:** A text-based interface keeps the project focused on programming fundamentals and core logic.

• **Input validation:** Validation prevents common invalid entries from reaching the priority calculation.

• **In-memory records:** A list of topic dictionaries provides a simple data model appropriate for the current scope.

12. Challenges and Learning Outcomes

The project required handling date calculations, multiple input-validation cases, rule-based scoring, sorting of topic records and state changes when a topic is completed. It also required coordinating several features through a menu-driven console interface.

• Using Python functions and modular program organization.

• Working with lists and dictionaries to represent application data.

• Using conditional logic and loops to implement rules and menus.

• Using datetime for deadline calculations.

• Using sorting to produce a priority-based study order.

• Applying exception handling and validation to user input.

• Designing and executing functional tests.

13. Limitations

• Topics are stored only in memory and are lost after the application exits.

• The application does not provide a graphical user interface.

• There is no database or user-account system.

• There is no calendar integration.

14. Future Enhancements

• JSON or database persistence.

• Graphical user interface.

• Calendar integration and reminders.

• Visual progress charts.

• Study timers.

• User accounts and personalized study profiles.

15. Conclusion

The Study Session Optimizer demonstrates how fundamental Python programming concepts can be combined to build a useful rule-based application. The system accepts study information, prioritizes pending work, supports completion and search operations, and reports progress through a simple console interface. The documented tests and execution screenshots provide evidence of the implemented functionality.

16. References

• VITyarthi – Build Your Own Project instructions.

• CSE1021 – Problem Solving and Programming course materials.

• Python standard-library documentation.